FFI System Improvement Plan

Version: 1.0 **Date**: 2026-02-25 **Status**: PROPOSED **Based On**: FFI_SYSTEM_AUDIT.md

---

User-Facing Summary

What's Changing

The Canopy FFI system is receiving a major safety upgrade. After this work:

1. **Type Safety at Runtime** - FFI calls will be validated at runtime. If JavaScript returns the wrong type, you'll get a clear error instead of silent corruption.
2. **Automatic Marshalling** - No more manually constructing `{ $: 'Ok', a: value }` in JavaScript. Canopy will generate helper functions that handle this for you.
3. **Better Error Messages** - When FFI fails, you'll see exactly which function, what type was expected, and what was actually received.
4. **Contract Validation** - New `canopy ffi validate` command checks that your JavaScript matches your Canopy declarations before you ship.
5. **Capability System That Works** - The capability types (`UserActivated` , `Initialized` , etc.) will actually be enforced at runtime.
6. **Binding Generation** - New `canopy ffi generate` command can create FFI bindings from TypeScript definitions.

Migration Guide

Existing FFI code will continue to work but will emit deprecation warnings. To upgrade:

```
# Check your FFI for issues
canopy ffi validate

# Generate new-style bindings
canopy ffi upgrade src/MyFFI.can

# Run with strict validation (recommended)
canopy make --ffi-strict
```

New FFI Syntax (Optional)

```
-- Old style (still works, deprecated)
foreign import javascript "external/audio.js" as Audio

-- New style (recommended)
foreign import javascript "external/audio.js" as Audio
    with validation = strict
    with marshalling = auto
```

Detailed Implementation Plan

Phase 1: Foundation (Weeks 1-2)

Change 1.1: Runtime Type Validator Infrastructure

What: Create a JavaScript runtime library that validates values against Canopy type schemas.

Files to Create:

- `packages/canopy-core/src/FFI/Validator.hs` - Haskell code to generate validators
- `packages/canopy-core/js/ffi-runtime.js` - JavaScript runtime library
- `packages/canopy-core/src/FFI/Schema.hs` - Type schema representation

Implementation:

```haskell
-- FFI/Schema.hs
module FFI.Schema
  ( TypeSchema(..)
  , schemaFromFFIType
  , schemaToJson
  ) where

-- | JSON-serializable type schema for runtime validation
data TypeSchema
  = SchemaInt
  | SchemaFloat
  | SchemaBool
  | SchemaString
  | SchemaUnit
  | SchemaMaybe !TypeSchema
  | SchemaList !TypeSchema
  | SchemaResult !TypeSchema !TypeSchema
  | SchemaTask !TypeSchema !TypeSchema
  | SchemaRecord ![(Text, TypeSchema)]
  | SchemaTuple ![TypeSchema]
  | SchemaOpaque !Text !Text  -- module, type name
  | SchemaFunction ![TypeSchema] !TypeSchema
  deriving (Eq, Show)

-- | Convert FFIType to TypeSchema
schemaFromFFIType :: FFIType -> TypeSchema
schemaFromFFIType = \case
  FFIBasic "Int" -> SchemaInt
  FFIBasic "Float" -> SchemaFloat
  FFIBasic "Bool" -> SchemaBool
  FFIBasic "String" -> SchemaString
  FFIBasic "()" -> SchemaUnit
  FFIMaybe inner -> SchemaMaybe (schemaFromFFIType inner)
  FFIList inner -> SchemaList (schemaFromFFIType inner)
  FFIResult err val -> SchemaResult (schemaFromFFIType err) (schemaFromFFIType val)
  FFITask err val -> SchemaTask (schemaFromFFIType err) (schemaFromFFIType val)
  FFIRecord fields -> SchemaRecord (map (second schemaFromFFIType) fields)
```

```
  FFITuple types -> SchemaTuple (map schemaFromFFIType types)
  FFIOpaque name -> SchemaOpaque "" name
  FFIFunctionType params ret -> SchemaFunction (map schemaFromFFIType params)
(schemaFromFFIType ret)
```

```
// js/ffi-runtime.js
const CanopyFFI = {
  /**
   * Validate a value against a type schema
   * @throws {FFITypeError} if validation fails
   */
  validate(value, schema, path = 'value') {
    switch (schema.type) {
      case 'int':
        if (!Number.isInteger(value)) {
          throw new FFITypeError(`${path}: expected Int, got ${typeof value}`);
        }
        return value;

      case 'float':
        if (typeof value !== 'number') {
          throw new FFITypeError(`${path}: expected Float, got ${typeof value}`);
        }
        return value;

      case 'bool':
        if (typeof value !== 'boolean') {
          throw new FFITypeError(`${path}: expected Bool, got ${typeof value}`);
        }
        return value;

      case 'string':
        if (typeof value !== 'string') {
          throw new FFITypeError(`${path}: expected String, got ${typeof value}`);
        }
        return value;

      case 'maybe':
        if (value === null || value === undefined) {
          return { $: 'Nothing' };
        }
        return { $: 'Just', a: this.validate(value, schema.inner, `${path}.value`)
};

      case 'list':
        if (!Array.isArray(value)) {
          throw new FFITypeError(`${path}: expected List, got ${typeof value}`);
        }
        return this._arrayToList(value.map((v, i) =>
          this.validate(v, schema.inner, `${path}[${i}]`)
        ));
```

```
      case 'result':
        // Results must be explicitly constructed
        if (!value || (value.$ !== 'Ok' && value.$ !== 'Err')) {
          throw new FFITypeError(`${path}: expected Result, got
${JSON.stringify(value)}`);
        }
        if (value.$ === 'Ok') {
          return { $: 'Ok', a: this.validate(value.a, schema.ok, `${path}.Ok`) };
        }
        return { $: 'Err', a: this.validate(value.a, schema.err, `${path}.Err`) };

      case 'record':
        if (typeof value !== 'object' || value === null) {
          throw new FFITypeError(`${path}: expected Record, got ${typeof value}`);
        }
        const result = {};
        for (const [field, fieldSchema] of Object.entries(schema.fields)) {
          if (!(field in value)) {
            throw new FFITypeError(`${path}: missing field '${field}'`);
          }
          result[field] = this.validate(value[field], fieldSchema,
`${path}.${field}`);
        }
        return result;

      case 'opaque':
        // Opaque types - check constructor if available
        if (schema.constructor && !(value instanceof schema.constructor)) {
          throw new FFITypeError(`${path}: expected ${schema.name}, got
${value?.constructor?.name || typeof value}`);
        }
        return value;

      default:
        throw new FFITypeError(`Unknown schema type: ${schema.type}`);
    }
  },

  /**
   * Convert JavaScript array to Canopy List
   */
  _arrayToList(arr) {
    let list = { $: 'Nil' };
    for (let i = arr.length - 1; i >= 0; i--) {
      list = { $: 'Cons', a: arr[i], b: list };
    }
    return list;
  },

  /**
   * Convert Canopy List to JavaScript array
```

```
   */
  _listToArray(list) {
    const arr = [];
    while (list.$ === 'Cons') {
      arr.push(list.a);
      list = list.b;
    }
    return arr;
  }
};

class FFITypeError extends Error {
  constructor(message) {
    super(message);
    this.name = 'FFITypeError';
  }
}

// Export for use
if (typeof module !== 'undefined') {
  module.exports = { CanopyFFI, FFITypeError };
}
if (typeof window !== 'undefined') {
  window.CanopyFFI = CanopyFFI;
  window.FFITypeError = FFITypeError;
}
```

Validation:

- [ ] Unit tests for all schema types
- [ ] Property tests: `validate(generate(schema), schema)` never throws
- [ ] Property tests: `validate(wrongType, schema)` always throws
- [ ] Integration test with existing audio-ffi functions
- [ ] Performance benchmark: validation overhead < 1µs per call

---

Change 1.2: Validator Code Generation

What: Generate JavaScript validators from Canopy FFI declarations.

Files to Modify:

- `packages/canopy-core/src/Foreign/FFI.hs` - Add validator generation
- `packages/canopy-core/src/Generate/JavaScript.hs` - Emit validator imports

Files to Create:

- `packages/canopy-core/src/FFI/ValidatorGen.hs` - Validator generation logic

Implementation:

```
-- FFI/ValidatorGen.hs
module FFI.ValidatorGen
  ( generateValidators
```

```
  , generateFunctionValidator
  , ValidatorOutput(..)
  ) where

data ValidatorOutput = ValidatorOutput
  { voSchemas :: !Text      -- JSON schema definitions
  , voValidators :: !Text   -- JavaScript validator functions
  , voWrappers :: !Text     -- Wrapped FFI functions
  }

-- | Generate validators for all functions in an FFI module
generateValidators :: [JSDocFunction] -> ValidatorOutput
generateValidators functions =
  ValidatorOutput
    { voSchemas = generateSchemas functions
    , voValidators = generateValidatorFunctions functions
    , voWrappers = generateWrappers functions
    }

-- | Generate a single function validator
generateFunctionValidator :: JSDocFunction -> Text
generateFunctionValidator func =
  let name = jsDocFuncName func
      (inputTypes, outputType) = flattenFunction (jsDocFuncType func)
      inputSchemas = zipWith makeInputSchema [0..] inputTypes
      outputSchema = schemaToJson (schemaFromFFIType outputType)
  in Text.unlines
    [ "const _validate_" <> name <> " = {"
    , "  inputs: [" <> Text.intercalate ", " inputSchemas <> "],"
    , "  output: " <> outputSchema
    , "};"
    , ""
    , "function " <> name <> "_validated(" <> params <> ") {"
    , "  // Validate inputs"
    , Text.unlines (zipWith validateInput [0..] inputTypes)
    , "  // Call original function"
    , "  const _result = " <> name <> "(" <> params <> ");"
    , "  // Validate and marshal output"
    , "  return CanopyFFI.validate(_result, _validate_" <> name <> ".output, '" <>
name <> ".result');"
    , "}"
    ]
  where
    params = Text.intercalate ", " ["a" <> Text.pack (show i) | i <- [0..length
inputTypes - 1]]
    validateInput i typ =
      "  CanopyFFI.validate(a" <> Text.pack (show i) <> ", _validate_" <> name <>
".inputs[" <> Text.pack (show i) <> "], '" <> name <> ".arg" <> Text.pack (show i)
<> "');"
```

Validation:

- [ ] Generated validators compile without errors
- [ ] Generated validators catch type mismatches
- [ ] Generated validators pass valid inputs unchanged
- [ ] All 225 audio-ffi functions generate valid validators
- [ ] Backward compatible: old code works with warnings

Change 1.3: FFI Strict Mode Flag

What: Add `--ffi-strict` flag to enable runtime validation.

Files to Modify:

- `packages/canopy-terminal/src/Make.hs` - Add flag parsing
- `packages/canopy-core/src/Generate/Mode.hs` - Add FFI mode
- `packages/canopy-core/src/Generate/JavaScript.hs` - Conditional validation

Implementation:

```
-- Generate/Mode.hs
data FFIMode
  = FFILegacy      -- No validation (current behavior)
  | FFIStrict      -- Full runtime validation
  | FFIDebug       -- Validation + detailed logging
  deriving (Eq, Show)

data Mode
  = Dev (Maybe I.Interfaces) FFIMode
  | Prod FieldShortener ShortFieldNames FFIMode
```

Validation:

- [ ] `canopy make` (no flag) uses FFILegacy
- [ ] `canopy make --ffi-strict` uses FFIStrict
- [ ] `canopy make --ffi-debug` uses FFIDebug
- [ ] Generated JS includes validators only in Strict/Debug mode
- [ ] Performance test: Legacy mode has zero overhead

Phase 2: Automatic Marshalling (Weeks 3-4)

Change 2.1: Marshalling Helper Generation

What: Generate JavaScript helpers for constructing Canopy data structures.

Files to Create:

- `packages/canopy-core/src/FFI/Marshal.hs` - Marshalling code generation
- `packages/canopy-core/js/ffi-marshal.js` - Runtime marshalling helpers

Implementation:

```
// js/ffi-marshal.js
const CanopyMarshal = {
```

```
// Result constructors
Ok(value) {
  return { $: 'Ok', a: value };
},

Err(error) {
  return { $: 'Err', a: error };
},

// Maybe constructors
Just(value) {
  return { $: 'Just', a: value };
},

Nothing() {
  return { $: 'Nothing' };
},

// List constructors
List(array) {
  let list = { $: 'Nil' };
  for (let i = array.length - 1; i >= 0; i--) {
    list = { $: 'Cons', a: array[i], b: list };
  }
  return list;
},

Nil() {
  return { $: 'Nil' };
},

// Task constructors (for async operations)
TaskOk(value) {
  return Promise.resolve({ $: 'Ok', a: value });
},

TaskErr(error) {
  return Promise.resolve({ $: 'Err', a: error });
},

// Wrap a Promise into a Task
TaskFromPromise(promise, errorMapper = String) {
  return promise
    .then(value => ({ $: 'Ok', a: value }))
    .catch(error => ({ $: 'Err', a: errorMapper(error) }));
},

// Tuple constructors
Tuple2(a, b) {
  return { a, b };
},
```

```
  Tuple3(a, b, c) {
    return { a, b, c };
  },

  // Unit
  Unit() {
    return { $: 'Tuple0' };
  },

  // Capability wrappers
  UserActivated(value) {
    return { $: 'UserActivated', a: value };
  },

  Initialized(value) {
    return { $: 'Initialized', a: value };
  },

  // Error type constructors
  CapabilityError: {
    UserActivationRequired(msg) {
      return { $: 'UserActivationRequired', a: msg };
    },
    PermissionRequired(msg) {
      return { $: 'PermissionRequired', a: msg };
    },
    InitializationRequired(msg) {
      return { $: 'InitializationRequired', a: msg };
    },
    FeatureNotAvailable(msg) {
      return { $: 'FeatureNotAvailable', a: msg };
    }
  },

  // Unwrap capability types
  unwrap(wrapped) {
    return wrapped.a;
  }
};
```

Updated audio.js Example:

```
// BEFORE (manual, error-prone)
function createAudioContext(userActivation) {
  try {
    const ctx = new AudioContext();
    return { $: 'Ok', a: { $: 'Fresh', a: ctx } };
  } catch (e) {
    return { $: 'Err', a: { $: 'InitializationRequired', a: e.message } };
  }
}
```

```
// AFTER (using marshalling helpers)
function createAudioContext(userActivation) {
  try {
    const ctx = new AudioContext();
    return CanopyMarshal.Ok(CanopyMarshal.Initialized(ctx));
  } catch (e) {
    return
CanopyMarshal.Err(CanopyMarshal.CapabilityError.InitializationRequired(e.message));
  }
}
```

Validation:

- [ ] All Result patterns use Ok/Err helpers
- [ ] All Maybe patterns use Just/Nothing helpers
- [ ] All List patterns use List helper
- [ ] Generated code is smaller than manual construction
- [ ] TypeScript definitions available for marshalling helpers

---

Change 2.2: Automatic Unwrapping

What: Generate code to automatically unwrap capability types.

Implementation:

```
-- FFI/Marshal.hs
generateUnwrapper :: FFIType -> Maybe Text
generateUnwrapper = \case
  FFIOpaque name | isCapabilityWrapper name ->
    Just $ "CanopyMarshal.unwrap"
  _ -> Nothing

isCapabilityWrapper :: Text -> Bool
isCapabilityWrapper name = name `elem`
  [ "UserActivated"
  , "Initialized"
  , "Permitted"
  , "Available"
  ]
```

Generated Code:

```
// For: getCurrentTime : Initialized AudioContext -> Float
function getCurrentTime_validated(initializedContext) {
  // Auto-generated unwrapping
  const _arg0 = CanopyMarshal.unwrap(initializedContext);
  CanopyFFI.validate(_arg0, { type: 'opaque', name: 'AudioContext' }, 'arg0');

  const _result = getCurrentTime(_arg0);
```

```
  CanopyFFI.validate(_result, { type: 'float' }, 'result');
  return _result;
}
```

Validation:

- [ ] Capability types are automatically unwrapped
- [ ] Unwrapped values are validated
- [ ] Non-capability types pass through unchanged
- [ ] Nested capabilities handled correctly

Phase 3: Error System (Weeks 5-6)

Change 3.1: Unified Error Types

What: Define a standard set of FFI error types that JavaScript must use.

Files to Create:

- `packages/canopy-core/src/FFI/Error.hs` - Error type definitions
- `packages/canopy-core/js/ffi-errors.js` - JavaScript error helpers

Implementation:

```
-- FFI/Error.hs
module FFI.Error
  ( FFIErrorType(..)
  , standardErrorTypes
  , errorTypeToSchema
  ) where

-- | Standard FFI error types
data FFIErrorType
  = TypeError           -- Wrong type received
  | RangeError          -- Value out of range
  | StateError          -- Invalid state for operation
  | NotSupportedError   -- Feature not supported
  | SecurityError       -- Security constraint violated
  | PermissionError     -- Permission not granted
  | NetworkError        -- Network operation failed
  | TimeoutError        -- Operation timed out
  | AbortError          -- Operation aborted
  | QuotaError          -- Resource quota exceeded
  | CustomError Text    -- Application-specific error
  deriving (Eq, Show)

-- | Map JavaScript error names to standard types
standardErrorTypes :: Map Text FFIErrorType
standardErrorTypes = Map.fromList
  [ ("TypeError", TypeError)
  , ("RangeError", RangeError)
```

```
  , ("InvalidStateError", StateError)
  , ("NotSupportedError", NotSupportedError)
  , ("SecurityError", SecurityError)
  , ("NotAllowedError", PermissionError)
  , ("NetworkError", NetworkError)
  , ("TimeoutError", TimeoutError)
  , ("AbortError", AbortError)
  , ("QuotaExceededError", QuotaError)
  ]
```

```
// js/ffi-errors.js
const FFIErrors = {
  // Map JavaScript errors to standard FFI errors
  fromJSError(error) {
    const mapping = {
      'TypeError': 'TypeError',
      'RangeError': 'RangeError',
      'InvalidStateError': 'StateError',
      'NotSupportedError': 'NotSupportedError',
      'SecurityError': 'SecurityError',
      'NotAllowedError': 'PermissionError',
      'NetworkError': 'NetworkError',
      'TimeoutError': 'TimeoutError',
      'AbortError': 'AbortError',
      'QuotaExceededError': 'QuotaError'
    };

    const errorType = mapping[error.name] || 'CustomError';
    return {
      $: errorType,
      a: error.message,
      _jsError: error  // Preserve original for debugging
    };
  },

  // Create structured error
  create(type, message, details = {}) {
    return {
      $: type,
      a: message,
      ...details
    };
  },

  // Standard error constructors
  TypeError: (msg) => FFIErrors.create('TypeError', msg),
  RangeError: (msg) => FFIErrors.create('RangeError', msg),
  StateError: (msg) => FFIErrors.create('StateError', msg),
  NotSupportedError: (msg) => FFIErrors.create('NotSupportedError', msg),
  SecurityError: (msg) => FFIErrors.create('SecurityError', msg),
  PermissionError: (msg) => FFIErrors.create('PermissionError', msg),
```

```
  NetworkError: (msg) => FFIErrors.create('NetworkError', msg),
  TimeoutError: (msg) => FFIErrors.create('TimeoutError', msg),
  AbortError: (msg) => FFIErrors.create('AbortError', msg),
  QuotaError: (msg) => FFIErrors.create('QuotaError', msg),
  CustomError: (type, msg) => FFIErrors.create(type, msg)
};
```

Validation:

- [ ] All standard error types defined in both Haskell and JavaScript
- [ ] Error mapping handles all common JavaScript errors
- [ ] Custom errors preserve type information
- [ ] Error messages include context (function name, argument)
- [ ] Canopy can pattern match on all error types

Change 3.2: Try/Catch Wrapper Generation

What: Generate safe wrappers that catch JavaScript exceptions.

Implementation:

```haskell
-- FFI/ValidatorGen.hs (addition)
generateSafeWrapper :: JSDocFunction -> Text
generateSafeWrapper func =
  let name = jsDocFuncName func
      (_, outputType) = flattenFunction (jsDocFuncType func)
      isResultType = case outputType of
        FFIResult _ _ -> True
        _ -> False
      isTaskType = case outputType of
        FFITask _ _ -> True
        _ -> False
  in if isResultType
     then generateResultWrapper name func
     else if isTaskType
     then generateTaskWrapper name func
     else generatePlainWrapper name func

generateResultWrapper :: Text -> JSDocFunction -> Text
generateResultWrapper name func = Text.unlines
  [ "function " <> name <> "_safe(" <> params <> ") {"
  , "  try {"
  , "    return " <> name <> "_validated(" <> params <> ");"
  , "  } catch (e) {"
  , "    if (e instanceof FFITypeError) {"
  , "      throw e;  // Re-throw type errors - they indicate bugs"
  , "    }"
  , "    return CanopyMarshal.Err(FFIErrors.fromJSError(e));"
  , "  }"
```

```
  , "}"
  ]
```

Validation:

- [ ] Result-returning functions catch and wrap errors
- [ ] Task-returning functions catch and wrap Promise rejections
- [ ] Plain functions propagate errors (or convert to Result)
- [ ] FFITypeError is never caught (indicates bug)
- [ ] Original stack traces preserved in debug mode

Phase 4: Capability System (Week 7)

Change 4.1: Implement CapabilityTracker

What: Create the missing CapabilityTracker runtime.

Files to Create:

- `packages/canopy-core/js/capability-tracker.js` - Runtime capability tracking

Implementation:

```
// js/capability-tracker.js
class CapabilityTracker {
  constructor() {
    this._userActivated = false;
    this._permissions = new Map();
    this._initialized = new Map();
    this._available = new Map();
    this._customCapabilities = new Map();

    // Track user activation
    this._setupUserActivationTracking();
  }

  _setupUserActivationTracking() {
    const activationEvents = ['click', 'keydown', 'touchstart', 'pointerdown'];

    const markActivated = () => {
      this._userActivated = true;
      // User activation expires after a short time
      setTimeout(() => {
        this._userActivated = false;
      }, 5000);
    };

    activationEvents.forEach(event => {
      document.addEventListener(event, markActivated, { passive: true });
    });
  }
```

```
  // User activation
  hasUserActivation() {
    // Also check navigator.userActivation if available
    if (navigator.userActivation) {
      return navigator.userActivation.isActive;
    }
    return this._userActivated;
  }

  requireUserActivation() {
    if (!this.hasUserActivation()) {
      throw FFIErrors.PermissionError('User activation required (click, tap, or
keypress)');
    }
  }

  // Permissions
  async requestPermission(name) {
    if (this._permissions.has(name)) {
      return this._permissions.get(name);
    }

    try {
      const result = await navigator.permissions.query({ name });
      const granted = result.state === 'granted';
      this._permissions.set(name, granted);
      return granted;
    } catch (e) {
      this._permissions.set(name, false);
      return false;
    }
  }

  hasPermission(name) {
    return this._permissions.get(name) === true;
  }

  requirePermission(name) {
    if (!this.hasPermission(name)) {
      throw FFIErrors.PermissionError(`Permission required: ${name}`);
    }
  }

  // Initialization tracking
  markInitialized(resource, instance) {
    this._initialized.set(resource, instance);
  }

  isInitialized(resource) {
    return this._initialized.has(resource);
  }
```

```
getInitialized(resource) {
  return this._initialized.get(resource);
}

requireInitialized(resource) {
  if (!this.isInitialized(resource)) {
    throw FFIErrors.StateError(`Resource not initialized: ${resource}`);
  }
  return this.getInitialized(resource);
}

// Feature availability
checkAvailability(feature) {
  if (this._available.has(feature)) {
    return this._available.get(feature);
  }

  let available = false;
  switch (feature) {
    case 'WebGL':
      available = !!window.WebGLRenderingContext;
      break;
    case 'WebGL2':
      available = !!window.WebGL2RenderingContext;
      break;
    case 'AudioContext':
      available = !!(window.AudioContext || window.webkitAudioContext);
      break;
    case 'ServiceWorker':
      available = 'serviceWorker' in navigator;
      break;
    case 'Clipboard':
      available = 'clipboard' in navigator;
      break;
    case 'Geolocation':
      available = 'geolocation' in navigator;
      break;
    default:
      available = feature in window;
  }

  this._available.set(feature, available);
  return available;
}

isAvailable(feature) {
  return this.checkAvailability(feature);
}

requireAvailable(feature) {
  if (!this.isAvailable(feature)) {
    throw FFIErrors.NotSupportedError(`Feature not available: ${feature}`);
```

```
    }
  }

  // Custom capabilities
  setCustomCapability(name, value = true) {
    this._customCapabilities.set(name, value);
  }

  hasCustomCapability(name) {
    return this._customCapabilities.get(name) === true;
  }

  requireCustomCapability(name) {
    if (!this.hasCustomCapability(name)) {
      throw FFIErrors.PermissionError(`Custom capability required: ${name}`);
    }
  }

  // Secure context
  isSecureContext() {
    return window.isSecureContext === true;
  }

  requireSecureContext() {
    if (!this.isSecureContext()) {
      throw FFIErrors.SecurityError('Secure context required (HTTPS)');
    }
  }
}

// Global instance
window.CapabilityTracker = new CapabilityTracker();
```

Validation:

- [ ] User activation tracking works on click/keypress
- [ ] Permission queries work for standard permissions
- [ ] Initialization tracking stores and retrieves resources
- [ ] Feature availability checks are accurate
- [ ] Secure context detection works
- [ ] All require* methods throw appropriate errors

Change 4.2: Capability Check Generation

What: Generate capability checks in FFI wrappers.

Files to Modify:

- `packages/canopy-core/src/Type/Capability.hs` - Use CapabilityTracker

Implementation:

```
-- Type/Capability.hs (updated)
generateCapabilityCheck :: Capability -> Text
generateCapabilityCheck = \case
  UserActivationCapability ->
    "CapabilityTracker.requireUserActivation();"

  PermissionCapability permission ->
    "CapabilityTracker.requirePermission('" <> permission <> "');"

  InitializationCapability resource ->
    "CapabilityTracker.requireInitialized('" <> resource <> "');"

  AvailabilityCapability feature ->
    "CapabilityTracker.requireAvailable('" <> feature <> "');"

  SecureContextCapability ->
    "CapabilityTracker.requireSecureContext();"

  CustomCapability name ->
    "CapabilityTracker.requireCustomCapability('" <> name <> "');"
```

Validation:

- [ ] Generated capability checks use CapabilityTracker
- [ ] Checks are inserted before function body
- [ ] Multiple capabilities checked in order
- [ ] Error messages identify which capability failed

Phase 5: CLI Tools (Week 8)

Change 5.1: FFI Validate Command

What: Add `canopy ffi validate` command.

Files to Create:

- `packages/canopy-terminal/src/FFI/Validate.hs` - Validation command

Implementation:

```
-- FFI/Validate.hs
module FFI.Validate
  ( run
  , ValidationResult(..)
  , ValidationIssue(..)
  ) where

data ValidationResult = ValidationResult
  { vrPassed :: !Int
  , vrWarnings :: ![ValidationIssue]
  , vrErrors :: ![ValidationIssue]
  }
```

```haskell
data ValidationIssue = ValidationIssue
  { viFile :: !FilePath
  , viFunction :: !Text
  , viSeverity :: !Severity
  , viMessage :: !Text
  , viSuggestion :: !(Maybe Text)
  }

data Severity = Warning | Error

run :: FilePath -> IO ValidationResult
run projectDir = do
  -- Find all FFI modules
  ffiModules <- findFFIModules projectDir

  -- Validate each module
  results <- traverse validateModule ffiModules

  -- Aggregate results
  pure $ mconcat results

validateModule :: FilePath -> IO ValidationResult
validateModule canopyFile = do
  -- Parse Canopy file for foreign imports
  foreignImports <- parseForeignImports canopyFile

  -- For each import, validate the JavaScript file
  results <- traverse (validateForeignImport canopyFile) foreignImports

  pure $ mconcat results

validateForeignImport :: FilePath -> ForeignImport -> IO ValidationResult
validateForeignImport canopyFile import_ = do
  let jsFile = resolveJSPath canopyFile (fiPath import_)

  -- Check JS file exists
  exists <- doesFileExist jsFile
  unless exists $
    pure $ singleError canopyFile (fiAlias import_) "JavaScript file not found"

  -- Parse JS file
  jsContent <- readFile jsFile
  case parseJavaScript jsContent of
    Left err ->
      pure $ singleError canopyFile (fiAlias import_) ("JS parse error: " <> err)
    Right ast -> do
      -- Extract JSDoc functions
      let jsFunctions = extractJSDocFunctions ast

      -- Check each declared function exists
      let declaredFunctions = getFunctionDeclarations canopyFile (fiAlias import_)
```

```
      issues <- traverse (validateFunction jsFunctions) declaredFunctions
      pure $ ValidationResult (length declaredFunctions - length (catMaybes issues))
[] (catMaybes issues)
```

CLI Interface:

```
# Validate all FFI in project
canopy ffi validate

# Validate specific file
canopy ffi validate src/AudioFFI.can

# Output formats
canopy ffi validate --format=json
canopy ffi validate --format=pretty  # default

# Strict mode (warnings are errors)
canopy ffi validate --strict
```

Validation:

- [ ] Detects missing JavaScript files
- [ ] Detects missing JSDoc annotations
- [ ] Detects type annotation syntax errors
- [ ] Detects function name mismatches
- [ ] Detects missing functions in JavaScript
- [ ] JSON output parseable by CI tools
- [ ] Exit code 0 for success, 1 for errors

Change 5.2: FFI Generate Command

What: Add `canopy ffi generate` for generating FFI bindings.

Files to Create:

- `packages/canopy-terminal/src/FFI/Generate.hs` - Binding generation

Implementation:

```
-- FFI/Generate.hs
module FFI.Generate
  ( run
  , GenerateOptions(..)
  ) where

data GenerateOptions = GenerateOptions
  { goInput :: !FilePath           -- TypeScript .d.ts or WebIDL
  , goOutput :: !FilePath          -- Output .can file
  , goModule :: !Text              -- Module name
  , goJsPath :: !FilePath          -- Path to JavaScript implementation
```

```
  }

run :: GenerateOptions -> IO ()
run opts = do
  -- Parse input file
  definitions <- parseDefinitions (goInput opts)

  -- Generate Canopy FFI module
  let canopyCode = generateCanopyModule (goModule opts) (goJsPath opts) definitions

  -- Generate JavaScript validators
  let validatorCode = generateValidators definitions

  -- Write outputs
  writeFile (goOutput opts) canopyCode
  writeFile (replaceExtension (goOutput opts) ".validators.js") validatorCode

  putStrLn $ "Generated " ++ goOutput opts
```

CLI Interface:

```
# Generate from TypeScript definitions
canopy ffi generate --from=audio.d.ts --module=AudioFFI --js=external/audio.js

# Generate from WebIDL
canopy ffi generate --from=WebAudio.webidl --module=WebAudio --
js=external/webaudio.js

# Generate validators only
canopy ffi generate --validators-only src/AudioFFI.can
```

Validation:

- [ ] Parses TypeScript .d.ts files
- [ ] Parses WebIDL files
- [ ] Generates valid Canopy syntax
- [ ] Generated module compiles without errors
- [ ] Validators catch type mismatches

Change 5.3: FFI Upgrade Command

What: Add `canopy ffi upgrade` to migrate existing FFI code.

Implementation:

```
# Upgrade a Canopy FFI module
canopy ffi upgrade src/AudioFFI.can

# Output:
# ✓ Added marshalling imports
```

```
# ✓ Generated validators for 225 functions
# ✓ Updated external/audio.js to use CanopyMarshal
# ⚠ 3 functions need manual review (see audio.js:142, :287, :456)
```

Validation:

- [ ] Adds required imports to Canopy file
- [ ] Generates validator file
- [ ] Updates JavaScript to use marshalling helpers
- [ ] Identifies functions that need manual fixes
- [ ] Creates backup of original files

Phase 6: Testing & Documentation (Weeks 9-10)

Change 6.1: Contract Testing Framework

What: Automated testing of FFI contracts.

Files to Create:

- `packages/canopy-terminal/src/FFI/ContractTest.hs` - Contract test runner

Implementation:

```haskell
-- FFI/ContractTest.hs
module FFI.ContractTest
  ( runContractTests
  , ContractTestResult(..)
  ) where

data ContractTestResult = ContractTestResult
  { ctrFunction :: !Text
  , ctrPassed :: !Bool
  , ctrInputsGenerated :: !Int
  , ctrFailures :: ![ContractFailure]
  }

data ContractFailure = ContractFailure
  { cfInput :: !Text         -- JSON representation of input
  , cfExpected :: !Text      -- Expected type
  , cfActual :: !Text        -- Actual value/type
  , cfError :: !(Maybe Text) -- Error message if threw
  }

runContractTests :: FilePath -> Int -> IO [ContractTestResult]
runContractTests canopyFile numRuns = do
  -- Parse FFI declarations
  ffiDecls <- parseForeignImports canopyFile

  -- For each function, generate test cases
  results <- traverse (testFunction numRuns) (concatMap getFFIFunctions ffiDecls)
```

```
  pure results

testFunction :: Int -> FFIFunction -> IO ContractTestResult
testFunction numRuns func = do
  -- Generate random inputs based on input types
  inputs <- replicateM numRuns (generateInputs (ffiFuncInputTypes func))

  -- Call function with each input
  results <- traverse (callAndValidate func) inputs

  -- Aggregate results
  let failures = [f | Left f <- results]

  pure $ ContractTestResult
    { ctrFunction = ffiFuncName func
    , ctrPassed = null failures
    , ctrInputsGenerated = numRuns
    , ctrFailures = failures
    }
```

CLI Interface:

```
# Run contract tests
canopy ffi test

# With more iterations
canopy ffi test --runs=1000

# Specific module
canopy ffi test src/AudioFFI.can

# Output contract test report
canopy ffi test --report=ffi-contract-report.html
```

Validation:

- [ ] Generates valid inputs for all FFI types
- [ ] Detects type contract violations
- [ ] Detects missing error handling
- [ ] Detects capability violations
- [ ] HTML report shows failures with reproducible cases

Change 6.2: Documentation Updates

What: Update all FFI documentation.

Files to Modify:

- `docs/website/src/guide/ffi.md` - Main FFI guide
- Add `docs/website/src/guide/ffi-migration.md` - Migration guide
- Add `docs/website/src/guide/ffi-best-practices.md` - Best practices

Documentation Outline:

1. **FFI Guide** (ffi.md)
   - Introduction to FFI
   - Basic syntax
   - Type mappings
   - Error handling
   - Capability system
   - Strict mode
2. **Migration Guide** (ffi-migration.md)
   - Why migrate
   - Step-by-step migration
   - Common issues
   - Before/after examples
3. **Best Practices** (ffi-best-practices.md)
   - DO: Use marshalling helpers
   - DO: Return Result for fallible operations
   - DO: Validate inputs
   - DON'T: Ignore errors
   - DON'T: Use raw data structures
   - Performance considerations

Validation:

- [ ] All code examples compile
- [ ] Migration guide tested on audio-ffi
- [ ] Best practices reviewed by team
- [ ] Tutorials have working demos

Phase 7: Performance & Polish (Weeks 11-12)

Change 7.1: Performance Optimization

What: Optimize validation overhead.

Optimizations:

1. **Schema Compilation**: Pre-compile schemas to optimized validators
2. **Trusted Mode**: Allow opting out of validation for performance-critical code
3. **Monomorphization**: Generate specialized validators for common types
4. **Caching**: Cache validation results for immutable values

Implementation:

```
// Compiled validator (fast path)
const _validate_getCurrentTime_input0 = CanopyFFI._compileValidator({
  type: 'opaque',
  name: 'AudioContext',
  constructor: window.AudioContext
```

```
});

function getCurrentTime_validated(arg0) {
  _validate_getCurrentTime_input0(arg0);  // Compiled, no schema lookup
  // ...
}
```

Validation:

- [ ] Validation overhead < 100ns for primitive types
- [ ] Validation overhead < 1µs for complex types
- [ ] Trusted mode has zero overhead
- [ ] Memory usage increase < 5% for large apps

Change 7.2: Error Message Polish

What: Make error messages helpful.

Example Error Messages:

```
FFI Type Error in AudioFFI.createOscillator

  Expected: Initialized AudioContext
  Received: { fake: "context" }

  The first argument to createOscillator must be an initialized
  AudioContext. Make sure you:

    1. Called createAudioContext first
    2. Passed the result (not the raw context)
    3. Handled any errors from createAudioContext

  See: https://canopy-lang.org/guide/ffi#audio-context
```

Validation:

- [ ] All error messages include function name
- [ ] All error messages show expected vs actual
- [ ] Complex errors include suggestions
- [ ] Errors link to documentation

Validation Matrix

Phase	Change	Unit Tests	Integration Tests	Property Tests	Manual QA
1	1.1 Validator Infrastructure	✓	✓	✓	✓

1	1.2 Validator Generation	✓	✓		✓
1	1.3 Strict Mode Flag	✓	✓		
2	2.1 Marshalling Helpers	✓	✓	✓	
2	2.2 Auto Unwrapping	✓	✓		
3	3.1 Unified Error Types	✓	✓		✓
3	3.2 Try/Catch Wrappers	✓	✓		
4	4.1 CapabilityTracker	✓	✓		✓
4	4.2 Capability Checks	✓	✓		
5	5.1 FFI Validate	✓	✓		✓
5	5.2 FFI Generate	✓	✓		✓
5	5.3 FFI Upgrade	✓	✓		✓
6	6.1 Contract Tests	✓	✓	✓	
6	6.2 Documentation				✓
7	7.1 Performance		✓		✓
7	7.2 Error Messages				✓

Success Criteria

Must Have (P0)

- [ ] All FFI calls validated at runtime (in strict mode)
- [ ] Marshalling helpers available and documented
- [ ] CapabilityTracker implemented and working
- [ ] `canopy ffi validate` command works
- [ ] audio-ffi upgraded and passing

Should Have (P1)

- [ ] `canopy ffi generate` works with TypeScript
- [ ] Contract testing framework complete
- [ ] Validation overhead < 1µs
- [ ] All documentation updated

Nice to Have (P2)

- [ ] WebIDL support
- [ ] IDE integration

- [ ] Performance profiling tools

Timeline

Week	Phase	Deliverables
1-2	Foundation	Validator infrastructure, schema types
3-4	Marshalling	Marshalling helpers, auto-unwrapping
5-6	Errors	Unified errors, try/catch wrappers
7	Capabilities	CapabilityTracker, capability checks
8	CLI	validate, generate, upgrade commands
9-10	Testing	Contract tests, documentation
11-12	Polish	Performance, error messages

Total: 12 weeks

Risk Assessment

Risk	Likelihood	Impact	Mitigation
Validation too slow	Medium	High	Compiled validators, trusted mode
Breaking existing code	High	High	Legacy mode, gradual migration
TypeScript parsing complex	Medium	Medium	Start with subset, expand
Browser compatibility	Low	Medium	Feature detection, polyfills
Team resistance	Low	High	Clear documentation, demos

Appendix: Upgraded audio-ffi Example

Before (current):

```
function createAudioContext(userActivation) {
    try {
        const ctx = new (window.AudioContext || window.webkitAudioContext)();
        return { $: 'Ok', a: { $: 'Fresh', a: ctx } };
    } catch (e) {
        if (e.name === 'NotSupportedError') {
            return { $: 'Err', a: { $: 'NotSupportedError', a: 'Web Audio API not
supported: ' + e.message } };
        } else {
            return { $: 'Err', a: { $: 'InitializationRequired', a: 'Failed to
create AudioContext: ' + e.message } };
        }
```

```
    }
}
```

After (upgraded):

```
/**
 * @name createAudioContext
 * @canopy-type UserActivated -> Result CapabilityError (Initialized AudioContext)
 * @capability user-activation
 */
function createAudioContext(userActivation) {
    // Capability check auto-generated
    CapabilityTracker.requireUserActivation();

    try {
        const ctx = new (window.AudioContext || window.webkitAudioContext)();
        CapabilityTracker.markInitialized('AudioContext', ctx);
        return CanopyMarshal.Ok(CanopyMarshal.Initialized(ctx));
    } catch (e) {
        return CanopyMarshal.Err(FFIErrors.fromJSError(e));
    }
}
```

Generated Wrapper (automatic):

```
function createAudioContext_validated(arg0) {
    // Input validation
    CanopyFFI.validate(arg0, { type: 'opaque', name: 'UserActivated' },
'createAudioContext.arg0');

    // Call implementation
    const result = createAudioContext_safe(arg0);

    // Output validation
    return CanopyFFI.validate(result, {
        type: 'result',
        err: { type: 'record', fields: { '$': 'string', 'a': 'string' } },
        ok: { type: 'opaque', name: 'Initialized', inner: 'AudioContext' }
    }, 'createAudioContext.result');
}
```